

Bangladesh University of Business and Technology
Department of Computer Science & Engineering

LAB 01: Introduction to ML - Python Basics

Duration: 3 Hours | Course: Introduction to Machine Learning | Spring 2026

Lab Objectives

By the end of this lab, students will be able to:

- Set up and verify a Python development environment
- Declare variables of different data types and perform type conversion
- Write for and while loops to process data
- Use Python lists and NumPy arrays for data manipulation
- Apply string operations relevant to text preprocessing
- Define functions with parameters, default arguments, and multiple return values
- Implement recursive functions (factorial, Fibonacci, sum)
- Read from and write to files using Python

Lab Schedule (3 Hours)

TIME	PHASE	TOPICS
0:00 – 0:20	Setup & Intro	Environment, attendance
0:20 – 1:00	Variables & Loops	Types, for/while, tasks
1:00 – 1:45	Arrays & Strings	Lists, NumPy, string ops
1:45 – 2:00	Break	Rest / Q&A
2:00 – 2:40	Functions & Files	Recursion, file read/write
2:40 – 3:00	Eval & Wrap-up	Task review, take-home

Documented By

Bijon Mallik
Lecturer, Department of CSE, BUBT
Mail: bijon@bubt.edu.bd
Cell: 01784595423
Google Scholar: [Bijon Mallik](#)
LinkedIn: [Bijon1161](#)

Part 1 - Environment Setup (0:00–0:20)

Installing Python: Open your terminal or VS Code terminal and run the following commands to verify your environment:

Steps:

1. First check python version in the terminal, if no version is showing then
2. Download: Visit python.org and download the latest Windows installer.
3. Run Installer: Open the .exe file.
4. Crucial Step: At the bottom, check Add Python to PATH.
5. Install: Click "Install Now."
6. Verify: Open Command Prompt (cmd) and type `python --version`

```
# Verify Python installation
python --version          # Expected: Python 3.x.x

# Install NumPy
python -m pip install numpy
```

Installing Jupyter:

1. Run “`python -m pip install jupyter`”
2. Run “`python -m notebook`” to open in browser
3. Run “start <https://colab.research.google.com>” for ML Projects

```
# Jupyter installation
python -m pip install jupyter          # Expected: Python 3.x.x

# Open Jupyter
python -m notebook

# Open Google Collab
start https://colab.research.google.com
```

Run First Program:

```
print("Hello Machine Learning")
```

Part 2 — Variables & Loops (0:20–1:00)

2.1 Variables and Data Types

Create a file called `basics.py` in your `ml_lab01/` folder. Type the following code:

```
# Variables & Data Types

name = input("Enter your name: ")
print("Welcome", name)

name = 'Alice'          # str
age = 21                 # int
score = 93.5             # float
passed = True            # bool

print(f'Student: {name}, Age: {age}, Score: {score}')
print(type(name), type(age), type(score))

# Type conversion
age_str = str(age)       # 21 → '21'
score_int = int(score)   # 93.5 → 93
print(f'Converted: {age_str!r} | {score_int!r}')
```

2.2 Loops

```
# For Loop - grade classifier

for i in range(5):
    print(i)

marks = [85, 72, 90, 65, 88]
for m in marks:
    if m >= 80:
        print(f'{m} → A')
    elif m >= 70:
        print(f'{m} → B')
    else:
        print(f'{m} → C')

# While Loop - compute average
total, i = 0, 0
while i < len(marks):
    total += marks[i]
    i += 1
print(f'Average: {total / len(marks):.2f}')

# enumerate
for idx, val in enumerate(marks):
    print(f'Student {idx+1}: {val}')
```

 Task 1 — Multiplication Table (10 min)

Write a program that asks the user for a number and prints its multiplication table from 1 to 10.

```
num = int(input('Enter a number: '))
for i in range(1, 11):
    print(f'{num} x {i} = {num * i}')
```

Part 3 — Arrays & Strings (1:00–1:45)

3.1 Python Lists and NumPy Arrays

```
# Python Lists

numbers = [10, 20, 30]
numbers.append(40)
print(numbers)

data = [3, 1, 4, 1, 5, 9, 2, 6]
print('Sorted  :', sorted(data))
print('Slice   :', data[2:5])
print('Reversed:', data[::-1])

# List comprehension - normalize to [0, 1]
normalized = [x / max(data) for x in data]
print('Norm    :', [round(v, 2) for v in normalized])

# NumPy Array
import numpy as np
arr = np.array([3, 1, 4, 1, 5, 9])
print('Mean:', np.mean(arr))
print('Std  :', np.std(arr))

matrix = np.array([[1,2,3],[4,5,6]])
print('Shape:', matrix.shape)  # (2, 3)
```

3.2 String Operations

```
text = '  Machine Learning is Fascinating!  '
print(text.strip())           # remove whitespace
print(text.lower())           # lowercase
print(text.replace('Fascinating', 'Powerful'))
print(text.split())           # tokenize
print('Learning' in text)     # True

# f-string formatting
n, v = 'Accuracy', 0.923
print(f'{n}: {v:.2f}')        # Accuracy: 0.92

# Word frequency (core NLP pattern)
sentence = 'the cat sat on the mat the cat'
freq = {}
for word in sentence.split():
```

```
freq[word] = freq.get(word, 0) + 1
print(freq)
```

Task 2 — Filter & Format (10 min, pair work)

Given a list of student names, print only names starting with 'A' in uppercase.

```
names = ['Alice', 'Bob', 'Arif', 'Clara', 'Anika']
result = [n.upper() for n in names if n.startswith('A')]
print(result)  # ['ALICE', 'ARIF', 'ANIKA']
```

Part 4 — Functions, Recursion & File I/O (2:00–2:40)

4.1 Functions

```
def add(a, b):
    return a + b

def classify_score(score: float) -> str:
    """Return grade label for a given score."""
    if score >= 90: return 'A+'
    elif score >= 80: return 'A'
    elif score >= 70: return 'B'
    else: return 'F'

scores = [95, 82, 74, 61]
for s in scores:
    print(f'Score {s} → {classify_score(s)}')

# Multiple return values
def stats(data, round_to=2):
    mean = sum(data) / len(data)
    variance = sum((x - mean)**2 for x in data) / len(data)
    return round(mean, round_to), round(variance**0.5, round_to)

mu, sigma = stats([10, 20, 30, 40])
print(f'Mean={mu}, Std={sigma}')
```

4.2 Recursion

Key rule: Every recursive function needs (1) a base case and (2) a recursive call that moves toward the base case.

```
# Factorial
def factorial(n):
    if n <= 1: return 1          # base case
    return n * factorial(n - 1)  # recursive call

print(factorial(5))  # 120
print(factorial(0))  # 1 (edge case)
```

```
# Fibonacci sequence
def fib(n):
    if n <= 1: return n
    return fib(n-1) + fib(n-2)

print([fib(i) for i in range(10)]) # [0,1,1,2,3,5,8,13,21,34]

# Recursive sum of a list
def recursive_sum(lst):
    if len(lst) == 0: return 0
    return lst[0] + recursive_sum(lst[1:])

print(recursive_sum([1, 2, 3, 4, 5])) # 15
```

4.3 File I/O

```
students = [('Alice',92), ('Bob',78), ('Clara',85), ('Daud',60)]

# Write
with open('results.txt', 'w') as f:
    f.write('Name,Score,Grade\n')
    for name, score in students:
        grade = classify_score(score)
        f.write(f'{name},{score},{grade}\n')

# Read
with open('results.txt', 'r') as f:
    for line in f:
        print(line.strip())

# Append
with open('results.txt', 'a') as f:
    f.write('Eve,91,A+\n')
```

Take-Home Assignment (Due: Next Session)

Student Report Generator

Build a complete program that does all of the following:

- Read student names and scores from a file input.txt
- Use a function to classify each score into a letter grade
- Use a recursive function to compute the sum of all scores
- Write a formatted summary report to report.txt
- **BONUS (+5 marks): Count how many students scored above average using list comprehension**

Quick Reference - Python Cheat Sheet

Variable	<code>x = 10 name = 'ML' pi = 3.14 flag = True</code>
Type check	<code>type(x) isinstance(x, int)</code>
For loop	<code>for i in range(10): for item in list:</code>
While loop	<code>while condition: break continue</code>
List ops	<code>[1,2,3] lst.append() lst[1:3] len(lst)</code>
List comp.	<code>[x*2 for x in lst if x > 0]</code>
NumPy	<code>np.array() .mean() .std() .shape</code>
String	<code>.strip() .split() .upper() .lower() .replace()</code>
Function	<code>def fn(x, default=0): return val1, val2</code>
File read	<code>with open('f.txt','r') as f: for line in f:</code>
File write	<code>with open('f.txt','w') as f: f.write(...)</code>